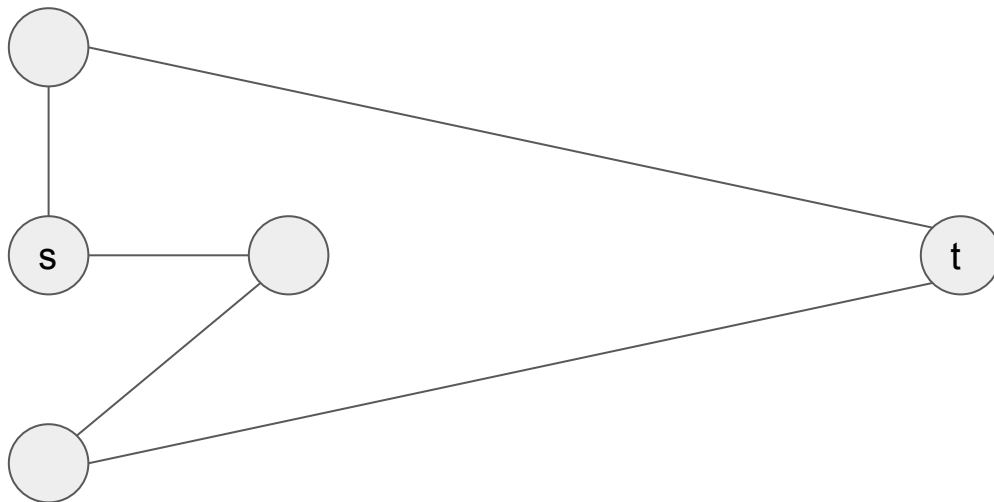


# Algoritmos gulosos

# Motivação

- Algoritmo guloso toma a decisão mais vantajosa na iteração (escolha gulosa)
  - Isso pode produzir uma solução final ruim (não fornece solução ótima)
  - Ex.: Objetivo: percorrer menor distância (euclidiana).
    - Escolha gulosa: seguir pela aresta com direção mais próxima do destino



# Motivação

- Poucos problemas possuem escolha gulosa que fornece solução ótima
  - Neste caso o algoritmo é simples e eficiente
  - Mas temos que provar que a escolha gulosa sempre fornece solução ótima
- Veremos estratégias de prova para os casos abaixo:
  - Solução é um subconjunto de um dado conjunto
    - Ex.: arestas em um caminho mais curto entre dois nós
  - Solução é uma forma de ordenar um conjunto de elementos
    - Ex.: ordem de atendimento de clientes de modo a minimizar o tempo médio

# Solução do problema de otimização é subconjunto

- Instância: um conjunto  $C$ , e parâmetros que definem subconjuntos válidos
- Solução: um subconjunto válido de  $C$
- Medida de sucesso: cada subconjunto tem um valor associado
- Objetivo: minimizar/maximizar a medida de sucesso
- Força bruta tem tempo exponencial (número de subconjuntos)

# Solução do problema de otimização é subconjunto

- Passos básicos
  - Em cada iteração um elemento  $u$  é escolhido de acordo com o critério guloso
  - Se a entrada de  $u$  na solução mantém a solução válida,  $u$  entra na solução
    - Caso contrário,  $u$  não entra na solução
  - O elemento  $u$  não é mais considerado nas próximas iterações
- Medida de progresso: número de elementos considerados
- Condição de parada: todos os elementos foram considerados

# Solução do problema de otimização é subconjunto

- É um algoritmo do tipo “estreitamento do espaço de busca”
  - Espaço de busca: todo subconjunto consistente com as decisões já tomadas, ou seja, que
    - contém os elementos que já foram colocados na solução;
    - não contém os elementos que foram considerados e não incluídos na solução
  - Em cada iteração consideramos um novo elemento  $u$  (escolhido pelo critério guloso)
    - Se  $u$  é colocado na solução, todo subconjunto sem  $u$  sai do espaço de busca
    - Se  $u$  não é colocado, todo subconjunto com  $u$  sai do espaço de busca
- Invariante do loop
  - Se existe solução válida, pelo menos uma solução ótima permanece no espaço de busca

# Solução do problema de otimização é subconjunto

- Estabelecendo o invariante do loop
  - Inicialmente nenhum elemento foi considerado (espaço completo)
- Pós-condições
  - Pelo invariante e condição de saída, a solução ótima é retornada
- Resta apenas mostrar que o código do loop mantém o invariante do loop
  - Inclusão/não inclusão da escolha gulosa na solução
    - mantém pelo menos uma solução ótima no espaço de busca

# Solução do problema de otimização é subconjunto

- Mantendo o invariante do loop
  - Assuma IL': se existe solução válida, alguma solução ótima ainda está no espaço de busca
  - Seja  $u$  a escolha gulosa
    - Se a inclusão de  $u$  torna a solução inválida, então IL' é satisfeito sem  $u$  na solução
    - Basta então considerar o caso em que  $u$  foi inserido na solução
  - Como provar que a inclusão de  $u$  na solução não remove toda solução ótima do espaço?
    - Podemos mostrar que toda solução ótima  $O$  restante pode ser adaptada para incluir  $u$ 
      - Sem piorar o valor da solução
      - Obs.: se inclusão de  $u$  melhora  $O$ , então  $O$  não é ótima (não existe o caso)



# Ex.: escalonamento de intervalos

- Instância:
  - $n$  intervalos. Cada intervalo  $i$  tem instante de início  $s[i]$  e de final  $f[i]$ , com  $0 \leq s[i] \leq f[i]$
- Solução: subconjunto de intervalos sem interseção
- Medida de sucesso: número de intervalos  $|S|$  na solução  $S$
- Objetivo: encontrar uma solução  $S$  que maximiza  $|S|$
  
- Vamos testar algumas escolhas gulosas
  - Para mostrar que não funciona, basta apresentar contra-exemplo

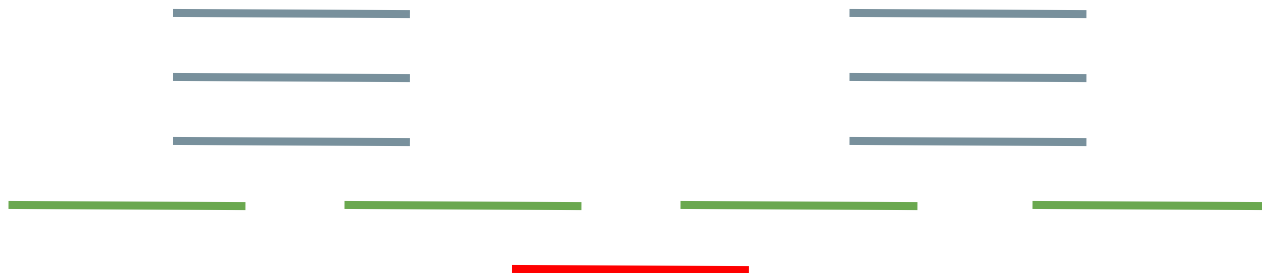
## Ex.: escalonamento de intervalos

- Escolha gulosa: menor intervalo
  - Não funciona



## Ex.: escalonamento de intervalos

- Escolha gulosa: o que tem interseção com menos intervalos
  - Não funciona



## Ex.: escalonamento de intervalos

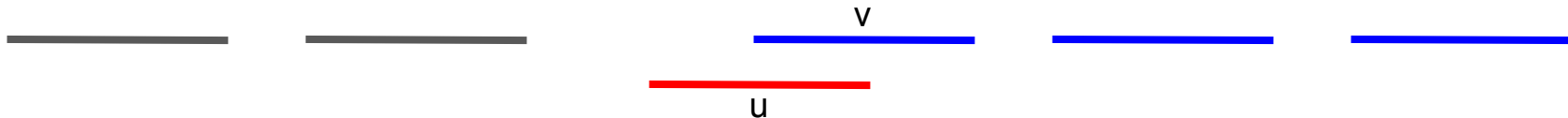
- Escolha gulosa: o que começa primeiro
  - Não funciona



## Ex.: escalonamento de intervalos

- Projeto: escolha que possa ser inserido em qq solução ótima, sem piorá-la
- Escolha gulosa: o que termina primeiro
  - Seja  $u$  o intervalo que termina primeiro (escolha gulosa)
  - Seja  $O$  uma solução ótima qualquer que
    - É compatível com as escolhas anteriores (inclusão/exclusão de intervalos)
    - Não contém o intervalo  $u$
  - Assuma que  $u$  não tem interseção com intervalos inseridos anteriormente
  - Como adaptar  $O$  para conter  $u$ , sem aumentar o número de intervalos?
    - Seja  $v$  o intervalo não considerado anteriormente que está em  $O$  e termina primeiro
    - Pelo critério guloso,  $u$  não termina depois de  $v$ 
      - Então  $u$  tem interseção com no máximo um intervalo em  $O$  (o intervalo  $v$ )
    - Podemos então trocar  $v$  por  $u$  em  $O$ , sem aumentar  $|O|$

$O$



## Ex.: escalonamento de intervalos



## Ex.: escalonamento de intervalos

Ordenado pelo instante de término



# Ex.: escalonamento de intervalos

algoritmo EscalonamentoIntervalos( $s[1..n]$ ,  $f[1..n]$ )

<pré-cond>:  $s[i]$  o instante de início de  $i$ ,  $f[i]$  instante de fim de  $i$ , para  $i = 1..n$ .

<pós-cond>: subconjunto de intervalos sem interseção de tamanho máximo

$S = \{\}$

ordene os intervalos em ordem não decrescente de  $f$

para cada intervalo  $i$  em ordem não decrescente de  $f$

se intervalo  $i$  não tem interseção com os intervalos em  $S$

$S = S \cup \{i\}$

retorne  $S$



# Ex.: escalonamento de intervalos

algoritmo EscalonamentoIntervalos( $s[1..n]$ ,  $f[1..n]$ )

<pré-cond>:  $s[i]$  o instante de início de  $i$ ,  $f[i]$  instante de fim de  $i$ , para  $i = 1..n$ .

<pós-cond>: subconjunto de intervalos sem interseção de tamanho máximo

$S = \{\}$

$fim\_ultimo = -\infty$

ordene os intervalos em ordem não decrescente de  $f$

para cada intervalo  $i$  em ordem não decrescente de  $f$

se  $fim\_ultimo \leq s[i]$

$S = S \cup \{i\}$

$fim\_ultimo = f[i]$

retorne  $S$

# Ex.: escalonamento de intervalos

algoritmo EscalonamentoIntervalos( $s[1..n]$ ,  $f[1..n]$ )

<pré-cond>:  $s[i]$  o instante de início de  $i$ ,  $f[i]$  instante de fim de  $i$ , para  $i = 1..n$ .

<pós-cond>: subconjunto de intervalos sem interseção de tamanho máximo

$S = \{\}$   $\theta(1)$

$fim\_ultimo = -\infty$   $\theta(1)$

ordene os intervalos em ordem não decrescente de  $f$   $\theta(n \log n)$

para cada intervalo  $i$  em ordem não decrescente de  $f$   $\theta(n)$

    se  $fim\_ultimo \leq s[i]$

$S = S \cup \{i\}$   $\theta(1)/\text{iteração}$

$fim\_ultimo = f[i]$

retorne  $S$   $\theta(1)$

Total:  $\theta(n \log n)$

Complexidade é dominada pela ordenação dos intervalos.

# Prioridade fixa vs adaptativa

- Prioridade fixa

- Ordem das escolhas gulosas não muda ao longo do algoritmo
  - Ex.: escalonamento de intervalos: ordem depende apenas do  $f$ , que permanece fixo
- Basta ordenar os elementos antes do laço

- Prioridade adaptativa

- Ordem das escolhas gulosas pode mudar
  - Pois é baseada em informação que muda durante a execução
- Ex.: algoritmo de Dijkstra
  - Escolha gulosa: menor  $d[v]$  (estimativa de distância) dentre os nós não finalizados
  - Valor  $d[v]$  pode mudar se alguma aresta  $(u,v)$  é relaxada
- Elementos devem então ser mantidos em uma heap

# Prioridade adaptativa

algoritmo GulosoAdaptativo(C)

<pré-cond>: C é um conjunto de objetos.

<pós-cond>: saída é um subconjunto ótimo de C.

coloque os elementos de C em uma heap H, onde a prioridade é o critério guloso

S = {} // elementos escolhidos

loop

<loop-inv>: se existe solução, pelo menos uma solução ótima é consistente com S.

saia quando H for vazia

u = H.extract\_min() // ou H.extract\_max(), dependendo da escolha gulosa

se S  $\cup$  {u} é uma solução válida

S = S  $\cup$  {u}

se necessário, atualize prioridades de elementos em H

retorne S

# Prioridade adaptativa - Ex.: Dijkstra

```
algoritmo Dijkstra(G,s,w)
  crie min-heap Q contendo o nó s com prioridade 0,
    e todos os outros nós com prioridade  $\infty$ 
  loop
    saia quando Q estiver vazia
    u = Q.extract_min()
    para cada vizinho v de u em G
      se d(v) > d(u) + w[u,v]
        d(v) = d(u) + w[u,v]
         $\pi(v)$  = u
        Q.update_priority(v,d(v))
```

# Solução do problema de otimização é uma ordem

- Instância: um conjunto  $C$ , e parâmetros que definem subconjuntos válidos
- Solução: uma ordem para os elementos de  $C$
- Medida de sucesso: cada ordem tem um valor associado
- Objetivo: minimizar/maximizar a medida de sucesso
- Força bruta:
  - Temos  $n!$  ordens possíveis para um conjunto com  $n$  elementos (inviável)

# Solução do problema de otimização é uma ordem

- Demonstração de corretude por argumento de troca
  - Assuma uma solução ótima  $O$  que não ordena os elementos como o guloso
  - Sejam  $a$  e  $b$  dois elementos consecutivos em  $O$  que não estão na ordem do guloso
  - Temos que mostrar que inverter as posições de  $a$  e  $b$  é viável e não piora a solução ótima  $O$
- Como todas as inversões em relação à ordem do guloso podem ser desfeitas
  - Sem piorar uma solução ótima
  - Concluimos que a ordem do guloso é ótima

# Ex.: Minimizar soma dos tempos de permanência

- Instância: tempo de atendimento  $t[i]$  para cada pessoa  $i = 1 \dots n$
- Solução: ordem de atendimento das pessoas
- Medida de sucesso: soma dos tempos de permanência
  - Permanência da  $k$ -ésima pessoa na solução:
    - Tempo de espera: soma dos tempos de atendimento das  $k-1$  pessoas na frente
    - Mais o tempo de atendimento da  $k$ -ésima pessoa
- Objetivo: minimizar a soma dos tempos de permanência

5      2      10      1      6      7

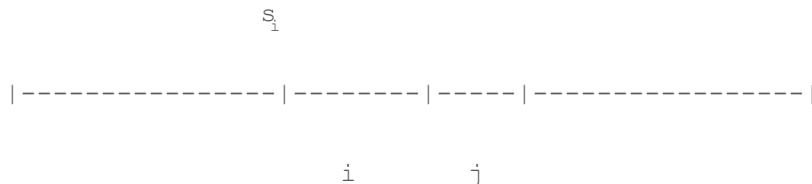
Exemplo: |-----|--|-----|---|-----|-----|

1      2      3      4      5      6



# Ex.: Minimizar soma dos tempos de permanência

- Considere uma solução ótima  $O$  onde  $i$  e  $j$  são consecutivos
  - E não estão na ordem do guloso



Mostrar que

$$(s_i + t[j] + t[i]) + (s_i + t[j]) \leq (s_i + t[i]) + (s_i + t[i] + t[j])$$

Conclusão:  $t[j] \leq t[i]$  (inverte quando  $t[j] \leq t[i]$ )

Guloso: primeiro quem tem o menor  $t$   
(ordem não decrescente de  $t$ )

- Inversão das posições de  $i$  e  $j$  mantém a solução  $O$  viável
- Temos que mostrar que esta inversão não piora  $O$ 
  - A inversão não afeta o tempo de permanência das outras pessoas
  - Permanência de  $i$  antes da troca:  $s_i + t[i]$
  - Permanência de  $j$  antes da troca:  $s_i + t[i] + t[j]$
  - Permanência de  $i$  depois da troca:  $s_i + t[j] + t[i]$
  - Permanência de  $j$  depois da troca:  $s_i + t[j]$